Full Stack: Spring Boot, JPA, REST & SQL

The framework stack — IoC, Boot, REST, persistence, SQL

10 topics · 95 flashcards · 50 quiz questions · 36 code examples

How to use this in 4 days

1. **Answer the quiz before you read.** Getting one wrong makes the answer stick; reading twice does not.
2. **Say it out loud.** Every topic ends with a scripted interview answer in the orange box. Reading it silently is not studying.
3. **Type the code.** Your hands remember what your eyes skim past.
4. **Days 1 and 2 are non-negotiable.** OOP, Collections and exceptions are where fresher rounds are won.
5. **Rehearse the project pitch out loud, three times.** It is the highest-leverage question in a fresher interview.

Day 4: Full Stack: Spring Boot, JPA, REST & SQL

The framework stack — IoC, Boot, REST, persistence, SQL

10 topics · 95 flashcards · 50 quiz questions · ~6.8 hours

> **Goal.** Be able to explain your project's architecture end to end: how a request travels from the client through a controller, service and repository to the database — and back.

Schedule

- Slot 1 (2h): Spring Core — IoC, DI, beans, scopes, annotations
- Slot 2 (2h): Spring Boot — auto-config, starters, properties, profiles
- Slot 3 (2h): REST controllers, DTOs, validation, exception handling
- Slot 4 (2.5h): JPA/Hibernate — entities, relationships, repositories, transactions
- Slot 5 (1.5h): SQL revision + project walkthrough out loud

1. Spring Core: IoC, DI & Beans Must-know 45 min

Two questions open almost every Spring interview: "What is IoC/DI?" and "What is a bean and what are its scopes?" Answer both crisply and the rest of the round goes smoothly.

Key points

- **IoC (Inversion of Control)**: you stop creating and wiring objects yourself; the container does it. "Don't call us, we'll call you."
- **DI (Dependency Injection)**: the mechanism — dependencies are *pushed into* a class rather than created inside it. Three types: **constructor** (recommended), setter, field.
- **Bean** = any object managed by the Spring container (instantiated, wired, and destroyed by it). The container is the `ApplicationContext`.
- Prefer **constructor injection** — it enables `final` fields, makes dependencies explicit, is immutable, and is easy to unit-test without Spring.
- Stereotypes: `@Component` (generic), `@Service` (business logic), `@Repository` (data access, translates DB exceptions), `@Controller`/`@RestController` (web).
- **Bean scopes**: singleton (default, one per container), prototype (new each request), request, session, application (web-only last three).

Reference

Annotation	Layer	What it does
@Component	Any	Generic Spring-managed bean
@Service	Service	Business logic — @Component with semantic meaning
@Repository	DAO	Data access + automatic exception translation to DataAccessException
@Controller	Web	MVC controller returning a view
@RestController	Web	@Controller + @ResponseBody → returns JSON
@Configuration	Config	Declares @Bean factory methods
@Autowired	Any	Injects a dependency (constructor, setter or field)
@Qualifier	Any	Disambiguates when multiple candidates exist

Code

IoC/DI: the before and after that explains everything

```
// WITHOUT Spring: you create and wire everything yourself (tightly coupled)
class OrderService {
   private OrderRepository repo = new JdbcOrderRepository();  // hard-coded!
   private PaymentClient pay    = new StripePaymentClient();  // hard-coded!
}
// Problems: can't swap implementations, can't unit test in isolation,
//           construction logic duplicated everywhere.

// WITH Spring: the container injects what's needed (loosely coupled)
@Service
public class OrderService {
   private final OrderRepository repo;
   private final PaymentClient pay;

   public OrderService(OrderRepository repo, PaymentClient pay) {  // constructor DI
       this.repo = repo;
       this.pay  = pay;
   }
}
// Spring sees the constructor, finds a bean of each type, and injects them.
// Fields can be final -> immutable and thread-safe by default.

@Repository
public class JpaOrderRepository implements OrderRepository { /* JPA impl */ }

// Swapping the implementation is now a one-line config change, and unit tests
// can pass a mock: new OrderService(mockRepo, mockPay);
```

Bean scopes in one place

```
// singleton (DEFAULT) - one instance shared by the whole container
@Service                                  // implicitly @Scope("singleton")
class StatsService { }

// prototype - a brand-new instance every time it's injected or fetched
@Component
@Scope("prototype")
class ReportBuilder { }

// web-aware scopes
@Component
@Scope(value = WebApplicationContext.SCOPE_REQUEST, proxyMode = ScopedProxyMode.TARGET_CLASS)
class RequestContext { }       // one per HTTP request

@Component
@Scope("session")
class ShoppingCart { }         // one per user session

// Defining beans manually with @Configuration + @Bean
@Configuration
class AppConfig {
   @Bean
   public ObjectMapper objectMapper() {
       return new ObjectMapper()
               .registerModule(new JavaTimeModule())
               .disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);
   }

   @Bean
   @Profile("dev")                  // only active when the dev profile is on
   public DataSource devDataSource() { /* H2 */ }

   @Bean
   @Profile("prod")
   public DataSource prodDataSource() { /* HikariCP + Postgres */ }
}

// Bean lifecycle: instantiate -> populate properties -> setBeanName ->
// BeanPostProcessors -> @PostConstruct init -> READY -> @PreDestroy on shutdown
@PostConstruct  public void init()   { /* after wiring */ }
@PreDestroy     public void cleanup(){ /* before removal */ }
```

Constructor vs field vs setter injection — and the @Autowired trap

```
// BEST: constructor injection (Spring 4.3+ doesn't even need @Autowired
// when there's a single constructor)
@Service
class GoodService {
   private final Repo repo;
   public GoodService(Repo repo) { this.repo = repo; }   // final, testable, explicit
}

// AVOID: field injection
@Service
class MehService {
   @Autowired private Repo repo;    // can't be final, hard to unit test,
                                    // hides dependencies, allows circular refs
}

// Resolving ambiguity when two beans implement the same interface
@Repository("jdbcRepo")  class JdbcRepo implements Repo { }
@Repository("jpaRepo")   class JpaRepo  implements Repo { }

@Service
class Uses {
   private final Repo repo;
   public Uses(@Qualifier("jpaRepo") Repo repo) { this.repo = repo; }
}
// Or mark one as @Primary: @Primary @Repository class JpaRepo implements Repo { }
```

What the container actually does on startup

```
@SpringBootApplication
public class Application {
   public static void main(String[] args) {
       // 1. Creates the ApplicationContext (the IoC container)
       // 2. Scans for @Component/@Service/@Repository/@Controller under this package
       // 3. Instantiates each bean definition (eagerly, by default)
       // 4. Injects dependencies (wiring)
       // 5. Calls @PostConstruct / InitializingBean
       // 6. Starts the embedded Tomcat on port 8080
       ApplicationContext ctx = SpringApplication.run(Application.class, args);

       // You can then pull beans out manually (useful in main()/tests):
       OrderService svc = ctx.getBean(OrderService.class);
   }
}

// Common startup failure to recognise:
// "No qualifying bean of type 'X' available"
//   -> the class isn't in a scanned package, or it lacks a stereotype annotation.
// "NoUniqueBeanDefinitionException"
//   -> two implementations; use @Qualifier or @Primary.
```

SAY IT LIKE THIS IN THE INTERVIEW

"IoC means the framework controls object creation and wiring instead of my code doing `new` everywhere. DI is how it's delivered — Spring injects dependencies, and I prefer constructor injection because it keeps fields final, makes dependencies explicit and makes unit testing trivial without Spring. A bean is any object managed by the container, singleton-scoped by default."

Flashcards

Q. What is IoC (Inversion of Control)?
A. The principle that object creation and lifecycle management is handled by a framework/container rather than your own code — 'don't call us, we'll call you'.

Q. What is Dependency Injection?
A. The implementation of IoC: dependencies are supplied to a class from the outside (constructor, setter or field) instead of being created inside it.

Q. Why prefer constructor injection?
A. Dependencies can be final/immutable, they're explicit and mandatory, circular dependencies fail fast, and you can unit test with plain mocks — no Spring context or reflection needed.

Q. What is a Spring bean?
A. Any object instantiated, assembled and managed by the Spring IoC container, defined via a stereotype annotation or an @Bean method.

Q. List the bean scopes.
A. singleton (default, one per container), prototype (new per injection), and for web apps: request, session, application, websocket.

Q. @Component vs @Service vs @Repository vs @Controller?
A. All are @Component specialisations. @Service marks business logic, @Repository marks data access and adds exception translation, @Controller/@RestController handle web requests.

Q. How do you resolve two beans of the same type?
A. Use @Qualifier("beanName") at the injection point, or mark one @Primary.

Q. @PostConstruct vs @PreDestroy?
A. @PostConstruct runs after dependency injection completes (init); @PreDestroy runs just before the bean is removed/container shuts down (cleanup).

Q. What is ApplicationContext?
A. The Spring IoC container interface: it holds bean definitions, manages their lifecycle, wires dependencies and provides features like events, i18n and resource loading.

Q. Is a singleton bean thread-safe?
A. Spring only guarantees one instance — it does NOT make it thread-safe. Keep singleton beans stateless (no mutable instance fields), or use synchronization/atomics.

Quiz

Q1. What is the relationship between IoC and DI?

[ ] A. They're the same thing
[x] B. IoC is the principle (container controls creation); DI is one way to implement it (dependencies are injected)
[ ] C. DI is the principle, IoC is the implementation
[ ] D. IoC is Spring-only, DI is Java EE-only

> **Why:** IoC is the broad design principle of inverting control; DI is the specific technique of supplying dependencies from outside.

Q2. Which injection type is recommended and why?

[ ] A. Field injection — least code
[x] B. Constructor injection — immutability, explicit dependencies, testability
[ ] C. Setter injection — most flexible
[ ] D. All are equally good

> **Why:** Constructor injection allows final fields, guarantees a fully-initialised object, and lets you instantiate the class in a unit test with no container.

Q3. What is the default bean scope in Spring?

[ ] A. prototype
[x] B. singleton
[ ] C. request
[ ] D. session

> **Why:** Singleton: one shared instance per Spring container. Important caveat: Spring does not make it thread-safe — keep singleton beans stateless.

Q4. You get `NoUniqueBeanDefinitionException`. What's the fix?

[ ] A. Delete one of the classes
[x] B. Use @Qualifier to name the bean you want, or mark one @Primary
[ ] C. Add @Scope("prototype")

[ ] D. Restart the server

> **Why:** Two beans satisfy the same type. Disambiguate at the injection point with @Qualifier, or designate a default with @Primary.

Q5. Why is `@Repository` more than just a marker?

[ ] A. It's identical to @Component
[x] B. It also enables persistence exception translation (vendor exceptions → Spring DataAccessException)
[ ] C. It creates the database schema
[ ] D. It makes methods transactional automatically

> **Why:** @Repository triggers a BeanPostProcessor that wraps the bean and translates technology-specific exceptions (e.g. SQLException) into Spring's consistent DataAccessException hierarchy. It does NOT add transactions.

Q6. What does @SpringBootApplication combine?

[x] A. @Configuration + @EnableAutoConfiguration + @ComponentScan
[ ] B. @Service + @Repository + @Controller
[ ] C. @Bean + @Autowired
[ ] D. @EntityScan + @EnableJpaRepositories

> **Why:** Those three annotations: it marks the class as a config source, enables Boot's auto-configuration, and triggers component scanning of the current package and below.

2. Spring Boot: Auto-configuration, Starters & Config Must-know 40 min

"What's the difference between Spring and Spring Boot?" — the answer is auto-configuration, starters, embedded servers and opinionated defaults. That's it, and it's a free question if you've rehearsed it.

Key points

- **Spring Boot = Spring + conventions**: auto-configuration, starter dependencies, embedded server, externalised config, production-ready actuator endpoints.
- **Auto-configuration**: Boot inspects the classpath and your beans, then configures sensible defaults. It's all conditional (`@ConditionalOnClass`, `@ConditionalOnMissingBean`).
- **Your beans always win** — auto-configuration backs off when you define your own (`@ConditionalOnMissingBean`).
- **Starters** are curated dependency bundles: `spring-boot-starter-web`, `-data-jpa`, `-security`, `-test`, `-validation`.
- Config lives in `application.properties` or `application.yml`; bind it with `@Value` or (better) `@ConfigurationProperties`.
- **Profiles** (`@Profile`, `spring.profiles.active`) separate dev/test/prod configuration.

Reference

Concern	Plain Spring	Spring Boot
Setup	Lots of XML/Java config by hand	Auto-configured from the classpath
Dependencies	You pick versions for everything	Starters bundle compatible versions
Server	Deploy a WAR to external Tomcat	Embedded Tomcat/Jetty/Undertow, java -jar
Config	Manual property plumbing	application.yml + @ConfigurationProperties
Monitoring	DIY	Spring Boot Actuator (/actuator/health, /metrics)

Code

A minimal Boot app and its project layout

```
@SpringBootApplication       // = @Configuration + @EnableAutoConfiguration + @ComponentScan
public class Application {
   public static void main(String[] args) {
       SpringApplication.run(Application.class, args);
   }
}

// Typical package structure (component scan starts at the root package):
// src/main/java/com/parvez/app/
//   Application.java          <- @SpringBootApplication (MUST be at the root)
//   controller/OrderController.java
//   service/OrderService.java
//   repository/OrderRepository.java
//   model/Order.java
//   dto/OrderRequest.java
//   exception/GlobalExceptionHandler.java
//   config/AppConfig.java
// src/main/resources/
//   application.yml
//   static/  templates/  data.sql
// src/test/java/...        <- JUnit tests
// pom.xml (Maven) or build.gradle (Gradle)
```

application.yml — the file you'll be asked about

```
server:
 port: 8081

spring:
 application:
   name: order-service
 profiles:
   active: dev
 datasource:
   url: jdbc:postgresql://localhost:5432/orders
   username: ${DB_USER}          # from an environment variable
   password: ${DB_PASS}
   driver-class-name: org.postgresql.Driver
 jpa:
   hibernate:
     ddl-auto: update           # none | validate | update | create | create-drop
   show-sql: true
   properties:
     hibernate:
       format_sql: true
   open-in-view: false          # IMPORTANT: avoid lazy-loading in the view layer

 hikari:
   maximum-pool-size: 10

logging:
 level:
   root: INFO
   com.parvez: DEBUG

---
spring:
 config:
   activate:
     on-profile: dev
 datasource:
   url: jdbc:h2:mem:testdb      # in-memory DB for local dev
```

Reading configuration: @Value vs @ConfigurationProperties

```
@Component
public class SimpleConfig {
   @Value("${app.name}")                    // single value
   private String appName;
   @Value("${app.timeout:5000}")            // with a DEFAULT if absent
   private int timeout;
   @Value("${JAVA_HOME}")                   // system env vars work too
   private String javaHome;
}

// BETTER for groups of related properties: type-safe, validated, IDE-completable
@ConfigurationProperties(prefix = "app.mail")     // needs @EnableConfigurationProperties
@Validated
public record MailProperties(
       @NotBlank String host,
       int port,
       @NotBlank String from,
       Duration timeout
) { }

// app.yml:
// app:
//   mail:
//     host: smtp.example.com
//     port: 587
//     from: noreply@example.com
//     timeout: 5s

// Profiles: activate per environment
// java -jar app.jar --spring.profiles.active=prod
// SPRING_PROFILES_ACTIVE=prod docker run ...
// Or per-bean: @Profile("dev") on a @Bean method
```

Starters and how auto-configuration decides

```
<!-- pom.xml: one starter pulls in a coherent, version-aligned set of libs -->
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-web</artifactId> <!-- Tomcat + Jackson + Spring MVC -->
</dependency>
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-data-jpa</artifactId><!-- Hibernate + Spring Data -->
</dependency>
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
   <groupId>com.h2database</groupId>
   <artifactId>h2</artifactId>
   <scope>runtime</scope>
</dependency>
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-test</artifactId>    <!-- JUnit5 + Mockito + AssertJ -->
   <scope>test</scope>
</dependency>

// How auto-config works (simplified):
@Configuration
@ConditionalOnClass(DataSource.class)          // only if the driver is on the classpath
@ConditionalOnMissingBean(DataSource.class)    // and I haven't defined my own
@EnableConfigurationProperties(DataSourceProperties.class)
public class DataSourceAutoConfiguration {
   @Bean
   public DataSource dataSource(DataSourceProperties props) { /* build one */ }
}
// See what Boot decided: add --debug, or hit /actuator/conditions.
```

SAY IT LIKE THIS IN THE INTERVIEW

"Spring Boot is Spring with opinionated defaults. Its auto-configuration inspects the classpath and conditionally configures beans — Tomcat because spring-boot-starter-web is present, a DataSource because a JDBC driver is — but it backs off the moment I define my own bean. Starters bundle compatible dependency versions, the app runs as a plain JAR with an embedded server, and config is externalised into application.yml bound with @ConfigurationProperties and separated by profiles."

Flashcards

Q. Spring vs Spring Boot?
A. Boot adds auto-configuration, starter dependencies, an embedded server, externalised config and Actuator — so you write far less boilerplate and run with `java -jar`.

Q. What is auto-configuration?
A. Boot inspects the classpath and existing beans, then applies @Conditional configuration (e.g. @ConditionalOnClass, @ConditionalOnMissingBean) to set up sensible defaults.

Q. How do you override an auto-configured bean?
A. Just define your own bean — auto-configuration classes use @ConditionalOnMissingBean and back off automatically.

Q. What is a Spring Boot starter?
A. A curated Maven/Gradle dependency bundle that brings in all the libraries needed for a feature with compatible versions (e.g. spring-boot-starter-web, -data-jpa, -security, -test).

Q. @Value vs @ConfigurationProperties?
A. @Value injects one property (good for single values). @ConfigurationProperties binds a whole typed, validated group — better for related settings and supports records.

Q. How do you separate dev and prod config?
A. Profiles: @Profile on beans, spring.profiles.active in config or as a runtime arg, plus profile-specific files (application-dev.yml) or --- documents in one YAML file.

Q. What is Spring Boot Actuator?
A. Production monitoring endpoints: /actuator/health, /info, /metrics, /loggers, /env, /beans, /conditions — usually secured and partially exposed.

Q. What does @SpringBootApplication combine?
A. @Configuration + @EnableAutoConfiguration + @ComponentScan.

Q. Why keep the main class at the root package?
A. @ComponentScan scans the package of the main class and everything below it — a class outside that tree simply won't be picked up (a very common 'bean not found' bug).

Q. What does spring.jpa.hibernate.ddl-auto=update do?
A. Hibernate updates the schema to match entities on startup. Fine for dev; in production prefer `validate` or `none` and manage migrations with Flyway/Liquibase.

Quiz

Q1. How does Spring Boot decide whether to configure a DataSource?

[ ] A. It always creates one
[x] B. Conditional annotations — e.g. @ConditionalOnClass(DataSource) and @ConditionalOnMissingBean — based on the classpath and your beans
[ ] C. You must always configure it manually
[ ] D. It reads a properties file only

> **Why:** Auto-configuration classes are loaded from META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports and guarded by @Conditional* annotations.

Q2. You define your own DataSource bean. What does Boot do with its auto-configured one?

[ ] A. Creates both, causing a conflict
[x] B. Backs off — @ConditionalOnMissingBean means yours wins
[ ] C. Throws an exception
[ ] D. Randomly picks one

> **Why:** That's the escape hatch that makes auto-configuration usable: conventions first, but your explicit definitions always take precedence.

Q3. Which dependency gives you an embedded Tomcat?

[ ] A. spring-boot-starter-data-jpa
[x] B. spring-boot-starter-web
[ ] C. spring-boot-starter-security
[ ] D. spring-core

> **Why:** spring-boot-starter-web pulls in spring-webmvc plus spring-boot-starter-tomcat, so the app can run standalone as a JAR.

Q4. How do you bind a group of properties to a typed object?

[ ] A. @Value on each field
[x] B. @ConfigurationProperties(prefix = "...")
[ ] C. @Autowired
[ ] D. @PropertySource only

> **Why:** @ConfigurationProperties binds and converts a whole prefix into a bean (works nicely with records), and supports validation with @Validated.

Q5. Which Actuator endpoint is typically used by orchestrators to check if the app is up?

[ ] A. /actuator/env
[x] B. /actuator/health

[ ] C. /actuator/beans

[ ] D. /actuator/shutdown

Why: /actuator/health is the standard liveness/readiness probe used by Kubernetes, load balancers and monitoring tools.

3. Spring MVC & REST Controllers Must-know 50 min

This is what you'll actually write on the job, so interviews go deep: mapping annotations, path/query/body parameters, status codes, validation, and global exception handling.

Key points

- **Request flow**: Client → DispatcherServlet (front controller) → HandlerMapping → Controller → Service → Repository → DB, then back through HttpMessageConverters (Jackson) → JSON response.
- `@RestController` = `@Controller` + `@ResponseBody` (return values are serialized to JSON, not resolved as views).
- `@PathVariable` = from the URL path; `@RequestParam` = query string; `@RequestBody` = JSON body; `@RequestHeader` = headers.
- **DTO pattern**: never expose entities directly — map to a request/response DTO so you control the API contract and avoid leaking internals or creating infinite JSON loops.
- **Validation**: `@Valid` + Jakarta Bean Validation annotations (`@NotNull`, `@NotBlank`, `@Size`, `@Email`, `@Min`) and a `@ExceptionHandler` for `MethodArgumentNotValidException`.
- Return `ResponseEntity<T>` when you need control over status codes/headers; return the DTO directly for the simple 200 case.

Reference

Annotation	Maps	Example
@GetMapping	HTTP GET (read)	@GetMapping("/users/{id}")
@PostMapping	HTTP POST (create)	@PostMapping("/users") — returns 201
@PutMapping	HTTP PUT (full update)	@PutMapping("/users/{id}")
@PatchMapping	HTTP PATCH (partial update)	@PatchMapping("/users/{id}")
@DeleteMapping	HTTP DELETE	@DeleteMapping("/users/{id}") — 204
@PathVariable	URL segment	/users/{id} → Long id
@RequestParam	Query string	?page=0&size=20
@RequestBody	JSON request body	Deserialized to a DTO
@Valid	Triggers validation	On @RequestBody or @ModelAttribute

Code

A complete REST controller (the shape to memorise)

```
@RestController
@RequestMapping("/api/users")
@RequiredArgsConstructor          // Lombok: generates the constructor for DI
public class UserController {

   private final UserService service;      // constructor injection

   @GetMapping
   public List<UserResponse> getAll(
           @RequestParam(defaultValue = "0") int page,
           @RequestParam(defaultValue = "20") int size) {
       return service.findAll(page, size);
   }

   @GetMapping("/{id}")
   public ResponseEntity<UserResponse> getById(@PathVariable Long id) {
       return service.findById(id)
               .map(ResponseEntity::ok)
               .orElseGet(() -> ResponseEntity.notFound().build());
   }

   @PostMapping
   public ResponseEntity<UserResponse> create(@Valid @RequestBody UserRequest req) {
       UserResponse created = service.create(req);
       URI location = ServletUriComponentsBuilder
               .fromCurrentRequest().path("/{id}")
               .buildAndExpand(created.id()).toUri();
       return ResponseEntity.created(location).body(created);   // 201 + Location
   }

   @PutMapping("/{id}")
   public UserResponse update(@PathVariable Long id,
                              @Valid @RequestBody UserRequest req) {
       return service.update(id, req);
   }

   @DeleteMapping("/{id}")
   @ResponseStatus(HttpStatus.NO_CONTENT)     // 204
   public void delete(@PathVariable Long id) { service.delete(id); }
}
```

DTOs, validation and why not to expose entities

```
// Request DTO - what the client may send, with validation rules
public record UserRequest(
       @NotBlank(message = "name is required") String name,
       @NotBlank @Email(message = "must be a valid email") String email,
       @Min(value = 18, message = "must be 18+") int age
) { }

// Response DTO - what the client receives (no password, no internal ids)
public record UserResponse(Long id, String name, String email, int age) {
   public static UserResponse from(User u) {
       return new UserResponse(u.getId(), u.getName(), u.getEmail(), u.getAge());
   }
}

// Why DTOs instead of returning the entity?
//  1. Security: never leak password hashes or internal fields by accident
//  2. Stability: changing the DB schema doesn't break the API contract
//  3. Loops: bidirectional JPA relations cause infinite JSON recursion
//     (User -> orders -> user -> orders -> ...) - @JsonIgnore is a band-aid,
//     DTOs are the fix
//  4. Validation and shaping (per-endpoint views of the same data)

// Mapping: write it by hand (as above), or use MapStruct / ModelMapper
@Mapper(componentModel = "spring")     // MapStruct generates the implementation
public interface UserMapper {
   UserResponse toResponse(User user);
   User toEntity(UserRequest request);
}
```

Global exception handling with @ControllerAdvice

```
@RestControllerAdvice           // applies to every @RestController
public class GlobalExceptionHandler {

   @ExceptionHandler(ResourceNotFoundException.class)
   @ResponseStatus(HttpStatus.NOT_FOUND)                // 404
   public ApiError handleNotFound(ResourceNotFoundException ex, WebRequest req) {
       return new ApiError(Instant.now(), 404, "Not Found",
                           ex.getMessage(), req.getDescription(false));
   }

   @ExceptionHandler(MethodArgumentNotValidException.class)
   @ResponseStatus(HttpStatus.BAD_REQUEST)              // 400
   public ApiError handleValidation(MethodArgumentNotValidException ex) {
       Map<String, String> errors = new HashMap<>();
       ex.getBindingResult().getFieldErrors()
         .forEach(e -> errors.put(e.getField(), e.getDefaultMessage()));
       return new ApiError(Instant.now(), 400, "Validation Failed",
                           errors.toString(), "/api");
   }

   @ExceptionHandler(Exception.class)                   // catch-all safety net
   @ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)    // 500
   public ApiError handleGeneric(Exception ex) {
       log.error("Unexpected error", ex);               // log, don't leak stack traces
       return new ApiError(Instant.now(), 500, "Internal Server Error",
                           "Something went wrong", "/api");
   }
}

public record ApiError(Instant timestamp, int status,
                       String error, String message, String path) { }
```

Status codes and the request pipeline

```
// The codes you must know:
//   200 OK            - successful GET/PUT
//   201 Created       - successful POST (+ Location header)
//   204 No Content    - successful DELETE
//   400 Bad Request   - invalid input / validation failure
//   401 Unauthorized  - not authenticated (no/invalid credentials)
//   403 Forbidden     - authenticated but not permitted
//   404 Not Found     - resource doesn't exist
//   409 Conflict      - state conflict (duplicate email etc.)
//   422 Unprocessable - syntactically valid but semantically wrong
//   500 Internal      - unexpected server error

// REQUEST PIPELINE (DispatcherServlet flow):
// 1. Client sends HTTP request
// 2. DispatcherServlet receives it (the single front controller)
// 3. HandlerMapping finds the controller method for the URL+verb
// 4. Argument resolvers bind @PathVariable / @RequestParam / @RequestBody
//    (Jackson converts JSON -> object via HttpMessageConverter)
// 5. Interceptors (@Aspect, filters, security) run around the handler
// 6. Controller -> Service (business logic, @Transactional) -> Repository
// 7. Return value converted to JSON by Jackson
// 8. Response sent with the appropriate status code

// Filter vs Interceptor vs AOP:
//   Filter    - servlet-level, runs before DispatcherServlet (Spring Security)
//   Interceptor - Spring MVC, pre/post handler (logging, auth checks)
//   AOP       - method-level, around advice (@Transactional, @Cacheable)
```

SAY IT LIKE THIS IN THE INTERVIEW

"A request hits the DispatcherServlet, which uses a HandlerMapping to find the controller method, binds parameters via HttpMessageConverters, runs the service layer and returns a value that Jackson serializes to JSON. I use constructor injection, return DTOs rather than entities so the API contract is stable and internal

fields never leak, validate incoming bodies with @Valid, and centralise error responses in a @RestControllerAdvice so the client always gets a consistent JSON error shape."

Flashcards

Q. What is DispatcherServlet?
A. Spring MVC's front controller: it receives every HTTP request and routes it to the right handler via HandlerMapping, then renders/converts the result.

Q. @Controller vs @RestController?
A. @RestController = @Controller + @ResponseBody. It serializes return values to JSON instead of resolving a view name.

Q. @PathVariable vs @RequestParam vs @RequestBody?
A. @PathVariable from the URL path (/users/{id}); @RequestParam from the query string (?page=0); @RequestBody from the JSON body, deserialized into an object.

Q. Why use DTOs instead of returning JPA entities?
A. They protect internal/sensitive fields, decouple the API from the DB schema, let you shape data per endpoint, and avoid infinite JSON recursion from bidirectional relationships.

Q. How do you validate a request body?
A. Annotate the DTO fields with Jakarta validation (@NotNull, @NotBlank, @Email, @Size) and mark the parameter @Valid; handle MethodArgumentNotValidException in a @ControllerAdvice.

Q. What status code for a successful POST?
A. 201 Created, ideally with a Location header pointing at the new resource (ResponseEntity.created(uri).body(dto)).

Q. What does @ControllerAdvice do?
A. It's a global, cross-cutting handler: @ExceptionHandler methods in it apply to all controllers, centralising error-to-HTTP-response mapping.

Q. Filter vs Interceptor?
A. Filters are servlet-level and run before DispatcherServlet (used by Spring Security). Interceptors are Spring MVC-level and run around the handler, with access to the handler/model.

Q. What makes a REST API 'RESTful'?
A. Statelessness, resource-based URLs (nouns, not verbs), correct HTTP verbs, proper status codes, HATEOAS (optional) and a uniform interface — commonly JSON over HTTP.

Q. What is the difference between PUT and PATCH?
A. PUT replaces the whole resource; PATCH applies a partial update.

Quiz

Q1. Which annotation extracts a value from `GET /users/42` where the method is mapped as `/users/{id}`?

[ ] A. @RequestParam
[x] B. @PathVariable
[ ] C. @RequestBody
[ ] D. @RequestHeader

Why: @PathVariable binds the {id} template variable from the URL path. @RequestParam would read ?id=42.

Q2. What is the correct response for creating a resource?

[ ] A. 200 OK
[x] B. 201 Created with a Location header
[ ] C. 204 No Content
[ ] D. 301 Moved Permanently

Why: 201 Created signals a new resource; including the Location header lets the client fetch it. 204 is for DELETE with no body.

Q3. Why should you avoid returning JPA entities directly from a controller?

[ ] A. It's slower

[x] B. It leaks internal fields, couples the API to the DB schema, and can cause infinite recursion with bidirectional relations

[ ] C. Jackson can't serialize entities

[ ] D. It breaks transactions

> **Why:** Entities carry fields you don't want public and relationships that Jackson will happily recurse into. DTOs give you a deliberate, stable contract.

Q4. Where should you handle validation failures?

[ ] A. Inside the try/catch in every controller method

[x] B. In a @ControllerAdvice with @ExceptionHandler(MethodArgumentNotValidException.class)

[ ] C. In the service layer only

[ ] D. Let them propagate to the client as stack traces

> **Why:** Centralising it gives consistent error JSON across the API and keeps controller methods clean — you return a field → message map of the violations.

Q5. How does JSON get converted to a Java object in @RequestBody?

[ ] A. Manually in the controller

[x] B. By an HttpMessageConverter — Jackson's MappingJackson2HttpMessageConverter

[ ] C. By the DispatcherServlet's view resolver

[ ] D. Java's built-in serializer

> **Why:** Spring selects an HttpMessageConverter based on the Content-Type; for application/json it's Jackson, which is on the classpath via spring-boot-starter-web.

Q6. Which is true about REST statelessness?

[ ] A. The server stores session data per client

[x] B. Each request must carry all the information needed; the server keeps no client session state

[ ] C. REST requires cookies

[ ] D. It only applies to GET

> **Why:** Statelessness is a REST constraint: no server-side session between requests. That's why JWTs (self-contained tokens) are a natural fit for REST auth.

4. JPA & Hibernate Must-know 60 min

The heaviest full-stack topic: entities, relationships, lazy vs eager, the N+1 problem, transactions and caching. If your interview mentions "your project used Hibernate", expect all of these.

Key points

- **JPA** is the specification (interfaces: `EntityManager`, annotations). **Hibernate** is the most common implementation. Spring Data JPA sits on top and generates repositories for you.
- An **entity** is a POJO mapped to a table: `@Entity`, `@Table`, `@Id`, `@GeneratedValue`, `@Column`.
- **Entity lifecycle**: Transient (new) → Managed/Persistent (inside a session) → Detached (session closed) → Removed.
- **Fetch types**: `@ManyToOne`/`@OneToOne` default to **EAGER**; `@OneToMany`/`@ManyToMany` default to **LAZY**. Always be explicit.
- **LazyInitializationException** happens when you touch a lazy collection outside an open session/transaction — fix with a fetch join, `@EntityGraph`, a DTO projection, or keeping the transaction open.
- **N+1 problem**: 1 query for the parents + N queries for each child. Fix with `JOIN FETCH`, `@EntityGraph`, or `@BatchSize`.

Reference

Annotation	Purpose	Notes
@Entity	Marks a class as a DB-mapped entity	Needs a no-arg constructor
@Table(name=...)	Maps to a specific table	Optional — defaults to the class name
@Id	Primary key	Required on every entity
@GeneratedValue	Key generation strategy	IDENTITY, SEQUENCE, AUTO (TABLE is discouraged)
@Column	Column mapping	name, nullable, unique, length
@OneToMany	One parent → many children	mappedBy names the owning side; LAZY by default
@ManyToOne	Many children → one parent	EAGER by default — set fetch = LAZY explicitly
@JoinColumn	Declares the FK column	Placed on the owning side
@Transactional	Declares a transaction	Use on the service layer, not the controller

Code

Entities and relationships, done properly

```
@Entity
@Table(name = "users")
public class User {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)   // auto-increment
   private Long id;

   @Column(nullable = false, length = 100)
   private String name;

   @Column(unique = true, nullable = false)
   private String email;

   @OneToMany(mappedBy = "user", cascade = CascadeType.ALL, orphanRemoval = true)
   private List<Order> orders = new ArrayList<>();      // LAZY by default

   public void addOrder(Order o) {                      // keep both sides in sync
       orders.add(o);
       o.setUser(this);
   }
   // no-arg constructor (required), getters, setters
}

@Entity
@Table(name = "orders")
public class Order {
   @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;

   @ManyToOne(fetch = FetchType.LAZY)          // EAGER by default - override it!
   @JoinColumn(name = "user_id")               // the FK column in `orders`
   private User user;

   @Enumerated(EnumType.STRING)                // store the name, not the ordinal!
   private OrderStatus status;

   @Column(name = "created_at", updatable = false)
   private LocalDateTime createdAt;

   @PrePersist
   protected void onCreate() { createdAt = LocalDateTime.now(); }
}

// Cascade types:
//   ALL          - propagate every operation
//   PERSIST      - save parent -> save children
//   MERGE        - update parent -> update children
//   REMOVE       - delete parent -> delete children
//   orphanRemoval = true  - deleting a child from the collection DELETES the row
```

Spring Data JPA repositories — queries without SQL

```
// No implementation needed - Spring generates it from the method name
public interface UserRepository extends JpaRepository<User, Long> {

   // Derived queries (the method name IS the query):
   Optional<User> findByEmail(String email);
   List<User> findByNameContainingIgnoreCase(String name);
   List<User> findByAgeGreaterThanEqualOrderByNameAsc(int age);
   boolean existsByEmail(String email);
   long countByStatus(OrderStatus status);
   void deleteByEmail(String email);

   // Custom JPQL - use @Param for named parameters
   @Query("SELECT u FROM User u JOIN FETCH u.orders WHERE u.id = :id")
   Optional<User> findByIdWithOrders(@Param("id") Long id);

   @Query("SELECT new com.app.dto.UserSummary(u.name, COUNT(o)) " +
          "FROM User u LEFT JOIN u.orders o GROUP BY u.name")
   List<UserSummary> findUserSummaries();      // DTO projection - no entity overhead

   // Native SQL when you need DB-specific features
   @Query(value = "SELECT * FROM users WHERE age > :age",
          nativeQuery = true)
   List<User> findAdultsNative(@Param("age") int age);

   // Modifying queries need @Modifying + @Transactional
   @Modifying
   @Query("UPDATE User u SET u.name = :name WHERE u.id = :id")
   int updateName(@Param("id") Long id, @Param("name") String name);

   // Paging and sorting come free
   Page<User> findAll(Pageable pageable);
}

// Using it:
Pageable page = PageRequest.of(0, 20, Sort.by("name").ascending());
Page<User> result = repo.findAll(page);
result.getContent(); result.getTotalPages(); result.getTotalElements();
```

The N+1 problem — recognise it, then fix it

```
// PROBLEM: 1 query for users + 1 query per user for their orders = N+1
List<User> users = userRepository.findAll();
for (User u : users) {
   System.out.println(u.getOrders().size());    // lazy load -> a query EACH time
}
// SQL log:  SELECT * FROM users;  SELECT * FROM orders WHERE user_id=1;
//           SELECT * FROM orders WHERE user_id=2;  ... (100 users -> 101 queries!)

// FIX 1: JOIN FETCH in JPQL (one query, orders populated already)
@Query("SELECT DISTINCT u FROM User u LEFT JOIN FETCH u.orders")
List<User> findAllWithOrders();

// FIX 2: @EntityGraph (declarative, no JPQL string)
@EntityGraph(attributePaths = {"orders"})
List<User> findAll();

// FIX 3: @BatchSize - fetch children in batches (e.g. 25 at a time)
@OneToMany(mappedBy = "user")
@BatchSize(size = 25)
private List<Order> orders;

// FIX 4: a DTO projection when you only need a few fields (fastest)
@Query("SELECT new com.app.dto.UserRow(u.id, u.name, SIZE(u.orders)) FROM User u")
List<UserRow> findUserRows();

// Spot it in logs: enable spring.jpa.show-sql=true or
// logging.level.org.hibernate.SQL=DEBUG and look for repeated similar queries.
```

Transactions, dirty checking and caching

```
@Service
@RequiredArgsConstructor
public class OrderService {
   private final OrderRepository repo;

   // @Transactional on the SERVICE layer (not the controller, not the repo)
   @Transactional
   public Order placeOrder(OrderRequest req) {
       Order o = new Order(req);          // managed entity
       o.setTotal(computeTotal(req));
       repo.save(o);                      // not strictly needed: dirty checking
       return o;
   }                                      // commit + flush on successful exit

   @Transactional(readOnly = true)        // optimisation: no dirty checking, no flush
   public List<Order> findAll() { return repo.findAll(); }

   @Transactional(rollbackFor = Exception.class)   // checked exceptions too
   public void cancel(Long id) throws CancellationException { /* ... */ }

   @Transactional(propagation = Propagation.REQUIRES_NEW)   // independent tx
   public void audit(String action) { /* always committed, even if the caller rolls back */ }
}

// DIRTY CHECKING: inside a transaction, Hibernate snapshots loaded entities and
// automatically UPDATEs any whose state changed at flush time — no save() needed.

// PROPAGATION: REQUIRED (default, join existing or create), REQUIRES_NEW,
// SUPPORTS, MANDATORY, NEVER, NOT_SUPPORTED, NESTED

// ISOLATION: DEFAULT, READ_UNCOMMITTED, READ_COMMITTED, REPEATABLE_READ, SERIALIZABLE
@Transactional(isolation = Isolation.READ_COMMITTED)

// CACHING:
//   1st level - the persistence context (per transaction/session), always on
//   2nd level - shared across sessions; enable with a provider (Ehcache/Caffeine)
//               @Cacheable + @org.hibernate.annotations.Cache(usage = READ_WRITE)
//   Query cache - caches result sets; use sparingly

// IMPORTANT: @Transactional doesn't work on private/self-invoked methods —
// it needs the Spring proxy, so calls must come from outside the bean.
```

LazyInitializationException — what causes it and the fixes

```
// CAUSE: the transaction/session closed before you touched the lazy collection
@Transactional
public User loadUser(Long id) {
   return repo.findById(id).orElseThrow();   // tx ends here, session closes
}
User u = service.loadUser(1L);
u.getOrders().size();     // LazyInitializationException: no Session

// FIX 1 (best): fetch what you need inside the transaction (JOIN FETCH / EntityGraph)
// FIX 2: use a DTO projection - map to the DTO before the tx ends
// FIX 3: @Transactional on the caller so the session spans the whole use case
// FIX 4 (AVOID): spring.jpa.open-in-view=true keeps the session open for the
//        whole web request - it hides the bug and holds DB connections longer.
//        Spring Boot warns about this on startup; set it to false.
```

SAY IT LIKE THIS IN THE INTERVIEW

"JPA is the specification and Hibernate is the implementation; Spring Data JPA generates repositories from interface method names. I map entities with @Entity/@Id/@GeneratedValue, set @ManyToOne to LAZY rather than accepting the EAGER default, and watch for the N+1 problem — which I fix with JOIN FETCH or @EntityGraph. Transactions belong on the service layer: Hibernate's dirty checking means I don't need to call save() on managed entities, and the commit happens when the transactional method exits."

Flashcards

Q. JPA vs Hibernate vs Spring Data JPA?
A. JPA is the specification (interfaces and annotations). Hibernate is the implementation. Spring Data JPA is a layer on top that generates repository implementations from interfaces.

Q. What are the entity lifecycle states?
A. Transient/New (not persisted), Managed/Persistent (attached to a session), Detached (session closed), Removed (scheduled for deletion).

Q. Lazy vs Eager fetching — what are the defaults?
A. @OneToMany and @ManyToMany default to LAZY; @ManyToOne and @OneToOne default to EAGER. Always declare fetch explicitly, and prefer LAZY.

Q. What is the N+1 problem and how do you fix it?
A. One query for the parents plus one per child collection. Fix with JOIN FETCH, @EntityGraph, @BatchSize, or a DTO projection.

Q. What is LazyInitializationException?
A. Accessing a lazy association after the session/transaction closed. Fix by fetching inside the transaction, using a DTO projection, or widening @Transactional — not by enabling open-in-view.

Q. Where should @Transactional go?
A. On the service layer (the unit of work), not on controllers or repositories. Use readOnly = true for queries. It won't apply to private or self-invoked methods because it's proxy-based.

Q. What is dirty checking?
A. Hibernate snapshots managed entities and automatically issues UPDATEs at flush time for any that changed — you don't need to call save() on a managed entity.

Q. GenerationType.IDENTITY vs SEQUENCE?
A. IDENTITY uses DB auto-increment (blocks JDBC batching). SEQUENCE uses a DB sequence and allows batching — generally preferred for performance with Hibernate.

Q. First-level vs second-level cache?
A. First-level is the persistence context (per transaction, always enabled). Second-level is shared across sessions and needs a provider like Ehcache/Caffeine plus @Cacheable.

Q. Why @Enumerated(EnumType.STRING)?
A. It stores the enum NAME instead of the ordinal. Ordinals break silently when you reorder or insert enum constants.

Q. What does mappedBy mean?
A. It marks the inverse (non-owning) side of a bidirectional relationship and names the field on the owning side that holds the foreign key.

Q. orphanRemoval vs cascade = REMOVE?
A. cascade REMOVE deletes children when the parent is deleted. orphanRemoval also deletes a child when it's removed from the parent's collection.

Quiz

Q1. What is the default fetch type of @ManyToOne?

[ ] A. LAZY
[x] B. EAGER
[ ] C. Depends on the DB
[ ] D. None

> **Why:** @ManyToOne and @OneToOne default to EAGER (a common performance surprise). @OneToMany and @ManyToMany default to LAZY. Always specify it explicitly.

Q2. You run `findAll()` on 100 users and then loop over each user's orders. How many queries?

[ ] A. 1
[ ] B. 2
[x] C. 101 (the N+1 problem)
[ ] D. 100

> **Why:** One query for the users plus one lazy load per user. Fix with JOIN FETCH, @EntityGraph, @BatchSize or a DTO projection.

Q3. Where is the best place for @Transactional?

[ ] A. On the controller
[x] B. On the service layer (the unit of work)
[ ] C. On the entity
[ ] D. On the DTO

> **Why:** The service method usually represents one business transaction. Controllers may fan out to several services, so the transaction boundary belongs in the service.

Q4. Why might @Transactional appear not to work?

[ ] A. It's deprecated
[x] B. The method is private, final, or called from within the same bean (self-invocation) — bypassing the proxy
[ ] C. The database doesn't support it
[ ] D. You imported the wrong package

> **Why:** Spring's transactions are AOP proxy-based, so only external calls through the proxy are intercepted. Also check you're using jakarta.transaction.Transactional or org.springframework.transaction.annotation.Transactional.

Q5. What is dirty checking?

[ ] A. Validating entities before insert
[x] B. Hibernate automatically detecting changed managed entities and issuing UPDATEs at flush time
[ ] C. Checking for SQL injection
[ ] D. Validating the schema

> **Why:** Because managed entities are tracked in the persistence context, mutating them inside a transaction is enough — an explicit save() is only needed for new (transient) entities.

Q6. How do you fetch a parent with its children in ONE query?

[ ] A. Set fetch = EAGER
[x] B. JOIN FETCH in JPQL or @EntityGraph
[ ] C. Call size() first
[ ] D. Use native SQL only

> **Why:** EAGER just hides the problem (it still issues queries, and always). JOIN FETCH or @EntityGraph retrieve parents and children together in a single SQL statement.

Q7. Which GenerationType allows JDBC batch inserts?

[ ] A. IDENTITY
[x] B. SEQUENCE
[ ] C. TABLE
[ ] D. AUTO

> **Why:** IDENTITY requires an immediate INSERT to obtain the key, which disables batching. SEQUENCE (or a pooled optimizer) lets Hibernate batch inserts efficiently.

5. SQL Essentials for Interviews Must-know 45 min

You will be asked to write SQL — usually a join, a group-by with having, and a 'second highest salary'-style query. Practise these until they're muscle memory.

Key points

- **Execution order**: FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT. (Knowing this explains why column aliases work in ORDER BY but not WHERE.)
- **JOINs**: INNER (matching rows), LEFT (all left + matches), RIGHT, FULL OUTER, CROSS. LEFT JOIN + `WHERE right.id IS NULL` finds unmatched rows.
- `WHERE` filters rows **before** grouping; `HAVING` filters groups **after**.
- **Indexes** speed up reads but slow writes. A composite index respects the leftmost-prefix rule (index on (a,b) helps `WHERE a=?` but not `WHERE b=?`).
- **Normalization**: 1NF (atomic values), 2NF (no partial dependency), 3NF (no transitive dependency). Denormalize deliberately for read performance.
- `NVL/COALESCE`, `CASE`, `DISTINCT`, `UNION` vs `UNION ALL` (UNION dedupes and is slower).

Code

The queries you'll actually be asked to write

```
-- Second highest salary (the classic)
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

-- Or with a window function / LIMIT
SELECT DISTINCT salary FROM employees ORDER BY salary DESC LIMIT 1 OFFSET 1;

-- Top earner per department
SELECT dept, name, salary FROM (
   SELECT e.*, ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC) rn
   FROM employees e
) t WHERE rn = 1;

-- Departments with more than 5 employees, by average salary
SELECT d.name, COUNT(e.id) AS headcount, AVG(e.salary) AS avg_salary
FROM departments d
JOIN employees e ON e.dept_id = d.id
WHERE e.active = true
GROUP BY d.name
HAVING COUNT(e.id) > 5
ORDER BY avg_salary DESC;

-- Employees with no department (LEFT JOIN ... IS NULL)
SELECT e.name FROM employees e
LEFT JOIN departments d ON e.dept_id = d.id
WHERE d.id IS NULL;

-- Duplicate emails
SELECT email, COUNT(*) FROM users GROUP BY email HAVING COUNT(*) > 1;

-- Employees earning more than their manager
SELECT e.name FROM employees e
JOIN employees m ON e.manager_id = m.id
WHERE e.salary > m.salary;

-- Pagination
SELECT * FROM orders ORDER BY created_at DESC LIMIT 20 OFFSET 40;
```

JOINs explained with a picture in words

```
-- Sample data
-- employees: (1, 'Ana', 10), (2, 'Ben', 20), (3, 'Cara', NULL)
-- departments: (10, 'Eng'), (20, 'Sales'), (30, 'HR')

-- INNER JOIN: only rows that match in BOTH -> Ana/Eng, Ben/Sales  (2 rows)
SELECT e.name, d.name FROM employees e INNER JOIN departments d ON e.dept_id = d.id;

-- LEFT JOIN: ALL left rows, NULLs where there's no match -> also Cara/NULL (3 rows)
SELECT e.name, d.name FROM employees e LEFT JOIN departments d ON e.dept_id = d.id;

-- RIGHT JOIN: all right rows -> also NULL/HR  (3 rows)
-- FULL OUTER JOIN: everything from both sides (not supported in MySQL!)

-- CROSS JOIN: cartesian product (3 x 3 = 9 rows) - usually a mistake
-- SELF JOIN: join a table to itself (employee -> manager)

-- JOIN vs WHERE filtering:
--   ON   -> condition applied while joining (affects which rows join)
--   WHERE-> applied after the join (a WHERE on a LEFT-joined column
--           effectively turns it into an INNER JOIN!)
```

Indexes, EXPLAIN and the things that kill performance

```
-- Create indexes
CREATE INDEX idx_emp_dept ON employees(dept_id);
CREATE UNIQUE INDEX idx_user_email ON users(email);
CREATE INDEX idx_order_date ON orders(created_at DESC);

-- Composite index: column order matters (leftmost prefix rule)
CREATE INDEX idx_name_age ON employees(last_name, age);
-- Helps:  WHERE last_name = 'X'                      -> yes
--         WHERE last_name = 'X' AND age = 30         -> yes
--         WHERE age = 30                             -> NO (skips the prefix)

-- See how the DB will run your query
EXPLAIN SELECT * FROM employees WHERE dept_id = 10;

-- Common index killers:
--   WHERE YEAR(created_at) = 2024        -> a function on the column blocks the index
--     FIX: WHERE created_at >= '2024-01-01' AND created_at < '2025-01-01'
--   WHERE name LIKE '%son'               -> a leading wildcard can't use the index
--   Implicit type conversion, OR across unindexed columns, SELECT * on huge tables

-- Best practices:
--   - Index foreign keys (they're your join and filter columns)
--   - Index the columns in WHERE/JOIN/ORDER BY
--   - Don't over-index: every index slows INSERT/UPDATE/DELETE
--   - Prefer covering indexes for hot queries
```

Useful patterns: CASE, COALESCE, EXISTS, window functions

```
-- CASE (SQL's if/else)
SELECT name,
      CASE WHEN salary > 100000 THEN 'High'
           WHEN salary >  50000 THEN 'Mid'
           ELSE 'Low' END AS band
FROM employees;

-- COALESCE: first non-null value (default handling)
SELECT name, COALESCE(bonus, 0) AS bonus FROM employees;

-- EXISTS is usually faster than IN for large subqueries
SELECT name FROM departments d
WHERE EXISTS (SELECT 1 FROM employees e WHERE e.dept_id = d.id);
-- EXISTS stops at the first match; IN materialises the whole result set.

-- Window functions (PostgreSQL/MySQL 8+/SQL Server)
SELECT name, dept, salary,
      RANK()       OVER (PARTITION BY dept ORDER BY salary DESC) AS rank_in_dept,
      SUM(salary)  OVER (PARTITION BY dept) AS dept_total,
      LAG(salary)  OVER (ORDER BY salary) AS previous_salary
FROM employees;

-- UPSERT (insert or update on conflict)
INSERT INTO users(email, name) VALUES ('a@b.c', 'Ana')
ON CONFLICT (email) DO UPDATE SET name = EXCLUDED.name;

-- Soft delete pattern + partial index
ALTER TABLE orders ADD COLUMN deleted_at TIMESTAMP NULL;
CREATE INDEX idx_orders_live ON orders(id) WHERE deleted_at IS NULL;
```

SAY IT LIKE THIS IN THE INTERVIEW

"I'm comfortable writing joins, group-by with having, subqueries and window functions. I know WHERE filters before grouping and HAVING after, that indexes speed reads but cost writes, and that a LEFT JOIN plus IS NULL finds unmatched rows. For slow queries I read the EXPLAIN plan first and look for full scans, functions on indexed columns and leading wildcards."

Flashcards

Q. WHERE vs HAVING?
A. WHERE filters individual rows before grouping (can't use aggregates). HAVING filters groups after GROUP BY (can use COUNT/SUM/AVG).

Q. What's the SQL query execution order?
A. FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT. (Which is why a SELECT alias works in ORDER BY but not WHERE.)

Q. INNER vs LEFT JOIN?
A. INNER returns only matching rows from both tables. LEFT returns every row from the left table, with NULLs where there's no match.

Q. How do you find rows with no match in another table?
A. LEFT JOIN and then `WHERE right_table.id IS NULL`.

Q. UNION vs UNION ALL?
A. UNION removes duplicates (extra sort/dedupe cost); UNION ALL keeps everything and is faster.

Q. What is the Nth-highest-salary pattern?
A. SELECT DISTINCT salary ORDER BY salary DESC LIMIT 1 OFFSET N-1, or a subquery with MAX(salary) < (SELECT MAX(...)), or DENSE_RANK() over the ordered salaries.

Q. How do indexes work and what's the trade-off?
A. An index (usually a B-tree) lets the DB find rows without a full scan: much faster reads, but every INSERT/UPDATE/DELETE must also maintain it, and it costs storage.

Q. What is the leftmost-prefix rule?
A. A composite index on (a, b) can serve queries filtering on a, or a+b, but not b alone — the leading column must be used.

Q. Why is `WHERE YEAR(date) = 2024` slow?
A. Applying a function to an indexed column prevents index usage; the DB must evaluate it per row. Use a range comparison instead.

Q. EXISTS vs IN?
A. EXISTS stops scanning at the first match and is generally better for large subqueries; IN materialises the full result set first.

Q. What are the normal forms?
A. 1NF: atomic column values, no repeating groups. 2NF: 1NF + no partial dependency on part of a composite key. 3NF: 2NF + no transitive dependency on non-key columns.

Q. What is a deadlock in a database?
A. Two transactions each hold a lock the other needs. The DB detects it and rolls one back (that's the deadlock error you see in logs) — prevent it with consistent lock ordering and short transactions.

Quiz

Q1. Which clause filters after GROUP BY?

[ ] A. WHERE
[x] B. HAVING
[ ] C. ORDER BY
[ ] D. LIMIT

> **Why:** HAVING filters groups and can reference aggregates (HAVING COUNT(*) > 5). WHERE filters rows before grouping.

Q2. You want all customers even if they have no orders. Which join?

[ ] A. INNER JOIN
[x] B. LEFT JOIN
[ ] C. CROSS JOIN
[ ] D. RIGHT JOIN only

> **Why:** LEFT JOIN keeps every row from the left (customers) and fills NULLs for missing matches (no orders).

Q3. What is the second highest salary query pattern?

[ ] A. SELECT salary FROM emp ORDER BY salary DESC LIMIT 2
[x] B. SELECT MAX(salary) FROM emp WHERE salary < (SELECT MAX(salary) FROM emp)
[ ] C. SELECT salary FROM emp WHERE rownum = 2
[ ] D. SELECT MIN(salary) FROM emp

> **Why:** The subquery excludes the maximum, so MAX of the remainder is the second highest. LIMIT 1 OFFSET 1 with DISTINCT is the other common form.

Q4. Does adding an index always make queries faster?

[ ] A. Yes, always
[x] B. No — it speeds reads but slows writes and consumes storage, and a badly-chosen index may not be used at all
[ ] C. Only for primary keys
[ ] D. Yes, if you index every column

> **Why:** Every write must update each index. Index the columns you actually filter, join and sort on — and verify with EXPLAIN.

Q5. Which of these prevents an index from being used?

[ ] A. WHERE status = 'A'
[x] B. WHERE UPPER(name) = 'ANA' (a function on the indexed column)
[ ] C. WHERE id = 42
[ ] D. WHERE created_at >= '2024-01-01'

Why: Wrapping the column in a function makes the predicate non-sargable. Store a normalized column, use a functional index, or rewrite as a range.

Q6. What does `COUNT(*)` vs `COUNT(col)` return?

[ ] A. Identical results always
[x] B. COUNT(*) counts all rows; COUNT(col) counts only non-NULL values of that column
[ ] C. COUNT(*) is slower and deprecated
[ ] D. COUNT(col) counts distinct values

Why: A classic trick question — COUNT(column) skips NULLs. Use COUNT(DISTINCT col) if you want unique values.

6. Maven, Build & Project Structure Core 25 min

Low difficulty, high frequency: "walk me through your pom.xml" and "what's Maven's build lifecycle?" — easy marks most candidates fumble.

Key points

- Maven uses a **POM** (`pom.xml`) with GAV coordinates: groupId, artifactId, version.
- **Standard layout**: `src/main/java`, `src/main/resources`, `src/test/java`, `target/` for build output.
- **Lifecycle phases**: `validate → compile → test → package → verify → install → deploy` (running a later phase runs all earlier ones).
- **Scope**: `compile` (default, everywhere), `provided` (JDK/container supplies it), `runtime`, `test`, `import`.
- **Dependency management**: Spring Boot's parent POM (or `spring-boot-dependencies` BOM) pins compatible versions so you don't specify them.
- Gradle is the alternative — faster, Groovy/Kotlin DSL, used by Android; same concepts, different syntax.

Code

A Spring Boot pom.xml, annotated

```
<project>
 <modelVersion>4.0.0</modelVersion>

 <parent>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-parent</artifactId>
   <version>3.2.5</version>          <!-- manages ALL dependency versions -->
   <relativePath/>
 </parent>

 <groupId>com.parvez</groupId>        <!-- your org, reverse domain -->
 <artifactId>order-service</artifactId><!-- the project name -->
 <version>0.0.1-SNAPSHOT</version>    <!-- SNAPSHOT = in development -->
 <packaging>jar</packaging>

 <properties>
   <java.version>17</java.version>
 </properties>

 <dependencies>
   <dependency>
     <groupId>org.springframework.boot</groupId>
     <artifactId>spring-boot-starter-web</artifactId>
     <!-- no <version> - inherited from the parent -->
   </dependency>
   <dependency>
     <groupId>org.projectlombok</groupId>
     <artifactId>lombok</artifactId>
     <scope>provided</scope>          <!-- compile-time only, not in the JAR -->
   </dependency>
   <dependency>
     <groupId>org.springframework.boot</groupId>
     <artifactId>spring-boot-starter-test</artifactId>
     <scope>test</scope>              <!-- test classpath only -->
   </dependency>
 </dependencies>

 <build>
   <plugins>
     <plugin>
       <groupId>org.springframework.boot</groupId>
       <artifactId>spring-boot-maven-plugin</artifactId>
       <!-- builds an executable "fat jar" with the embedded server -->
     </plugin>
   </plugins>
 </build>
</project>
```

Commands you'll actually run

```
mvn clean                # delete target/
mvn compile              # compile main sources
mvn test                 # run unit tests
mvn package              # build the JAR (runs tests too)
mvn package -DskipTests  # build without running tests
mvn spring-boot:run      # run the app directly (dev)
mvn clean install        # build + install into ~/.m2 for other projects
mvn dependency:tree      # see where a transitive dependency comes from
mvn versions:display-dependency-updates

java -jar target/order-service-0.0.1-SNAPSHOT.jar
java -jar app.jar --spring.profiles.active=prod

# Gradle equivalents
./gradlew bootRun
./gradlew build
./gradlew test
```

Lombok — the annotations to name-drop (and understand)

```
@Data                    // getters + setters + equals + hashCode + toString
@Getter @Setter          // just the accessors
@NoArgsConstructor       // JPA needs this
@AllArgsConstructor
@RequiredArgsConstructor  // one ctor for final/@NonNull fields -> constructor DI!
@Builder                 // User.builder().name("x").build()
@Slf4j                   // adds a static `log` field
@ToString(exclude = "password")
@EqualsAndHashCode(of = "id")   // use the business key, not every field

@Slf4j
@Service
@RequiredArgsConstructor      // <-- the standard Spring DI pattern
public class OrderService {
   private final OrderRepository repo;   // injected via the generated ctor

   public void place(Order o) {
       log.info("placing order {}", o.getId());
   }
}

// Caveat: Lombok is compile-time magic - it needs the IDE plugin and an
// annotation-processor entry. Records (Java 16) cover many of its use cases
// for immutable data, so prefer records for DTOs and Lombok for entities.
```

SAY IT LIKE THIS IN THE INTERVIEW

"Maven uses a POM with GAV coordinates and a fixed lifecycle: validate, compile, test, package, verify, install, deploy. The Spring Boot parent POM manages dependency versions so I don't have to align them myself. I scope dependencies properly — test for JUnit/Mockito, provided for Lombok — and the spring-boot-maven-plugin produces an executable fat JAR I can run with java -jar."

Flashcards

Q. What are Maven's GAV coordinates?
A. groupId (organisation, reverse domain), artifactId (project name), version (SNAPSHOT = in-development).

Q. List the Maven lifecycle phases.
A. validate, compile, test, package, verify, install, deploy. Running a later phase executes all the earlier ones.

Q. What does `mvn package` do vs `mvn install`?
A. package builds the artifact into target/. install does that AND installs it into the local ~/.m2 repository so other projects can depend on it.

Q. What is a Maven dependency scope?
A. compile (default, everywhere), provided (JDK/container provides it at runtime), runtime, test (test classpath only), system, import (BOM imports).

Q. What is a BOM / dependency management?
A. A 'bill of materials' POM that pins compatible versions of many libraries, so you declare dependencies without versions and never hit conflicts.

Q. What is a fat/uber JAR?
A. An executable JAR containing your classes plus all dependencies and an embedded server — built by spring-boot-maven-plugin, runnable with java -jar.

Q. Maven vs Gradle?
A. Maven uses declarative XML with a fixed lifecycle. Gradle uses a Groovy/Kotlin DSL, is generally faster with incremental builds and caching, and is the default for Android. Same core concepts.

Q. What does @RequiredArgsConstructor do and why is it popular in Spring?
A. Lombok generates a constructor for all final/@NonNull fields — which is exactly what constructor-based dependency injection needs, with zero boilerplate.

Quiz

Q1. Which Maven phase runs the tests and creates the JAR?

[ ] A. compile

[ ] B. test
[x] C. package
[ ] D. install

> **Why:** `package` runs the full lifecycle up to and including test, then builds the artifact in target/. `install` also copies it to your local repository.

Q2. Why use `spring-boot-starter-parent`?

[ ] A. It's required by Spring
[x] B. It provides dependency and plugin management with compatible versions, plus sensible defaults
[ ] C. It enables auto-configuration
[ ] D. It compiles Java faster

> **Why:** It's a parent POM that declares versions for the whole Boot ecosystem, so you omit <version> in your dependencies and avoid version conflicts.

Q3. Which scope should JUnit and Mockito use?

[ ] A. compile
[ ] B. provided
[x] C. test
[ ] D. runtime

> **Why:** Test-scoped dependencies are only on the test classpath and are never shipped in the final JAR.

Q4. What is a transitive dependency?

[x] A. A dependency of a dependency, pulled in automatically
[ ] B. A dependency in the test scope
[ ] C. An optional dependency
[ ] D. A circular dependency

> **Why:** Maven resolves the whole dependency tree automatically. Use `mvn dependency:tree` to see where something came from, or <exclusions> to drop an unwanted one.

7. Spring Security & JWT Edge 35 min

For a fresher they usually want the high-level flow, not deep configuration: the filter chain, authentication vs authorization, and how a JWT works.

Key points

- **Authentication** = who are you? **Authorization** = what may you do? (401 vs 403.)
- Spring Security is a chain of servlet **filters**; each request passes through them before reaching the controller.
- **JWT** (JSON Web Token) = header.payload.signature. It's **stateless** — the server stores nothing; the token carries the claims and is verified by signature.
- Never store secrets in a JWT — it's **Base64-encoded, not encrypted**. Anyone can decode the payload.
- **BCrypt** for password hashing — slow by design and salted. Never MD5/SHA or plain text, and never encrypt passwords (hash them).
- **CORS** (cross-origin) and **CSRF** (cross-site request forgery) are separate concerns: CSRF protection can be disabled for stateless JWT APIs, CORS must be configured explicitly.

Code

The JWT authentication flow (draw this in the interview)

```
1. POST /api/auth/login  { "username": "p", "password": "secret" }
       |
       v
2. Spring Security's filter chain intercepts it
       |
       v
3. AuthenticationManager -> DaoAuthenticationProvider -> UserDetailsService
  loads the user, BCrypt compares the hashed password
       |
       v
4. On success: generate a JWT signed with a secret (HS256)
  payload claims: subject (username), roles, issuedAt, expiration
       |
       v
5. Response: { "token": "eyJhbGciOi..." }
       |
       v
6. Client stores the token (memory / httpOnly cookie) and sends it on
  every request:  Authorization: Bearer eyJhbGciOi...
       |
       v
7. OncePerRequestFilter validates the signature + expiry on each request,
  loads the user, and populates the SecurityContext
       |
       v
8. @PreAuthorize("hasRole('ADMIN')") / the SecurityFilterChain's
  authorizeHttpRequests rules decide whether access is allowed
       |
       v
9. Controller runs (or 401/403 is returned)

A JWT is three Base64 parts joined by dots:  HEADER.PAYLOAD.SIGNATURE
 header   - { "alg": "HS256", "typ": "JWT" }
 payload  - { "sub": "parvez", "roles": ["ADMIN"], "exp": 1735689600 }
 signature- HMACSHA256(base64(header) + "." + base64(payload), secret)
```

A modern Spring Security 6 configuration

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity          // enables @PreAuthorize
@RequiredArgsConstructor
public class SecurityConfig {

   private final JwtAuthFilter jwtAuthFilter;
   private final UserDetailsService userDetailsService;

   @Bean
   public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
       return http
           .csrf(csrf -> csrf.disable())                // stateless JWT API
           .cors(cors -> cors.configurationSource(corsConfigurationSource()))
           .authorizeHttpRequests(auth -> auth
               .requestMatchers("/api/auth/**", "/actuator/health").permitAll()
               .requestMatchers(HttpMethod.GET, "/api/products/**").permitAll()
               .requestMatchers("/api/admin/**").hasRole("ADMIN")
               .anyRequest().authenticated()
           )
           .sessionManagement(s -> s
               .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
           .authenticationProvider(authenticationProvider())
           .addFilterBefore(jwtAuthFilter,
                            UsernamePasswordAuthenticationFilter.class)
           .build();
   }

   @Bean
   public PasswordEncoder passwordEncoder() {
       return new BCryptPasswordEncoder(12);       // cost factor 12
   }

   @Bean
   public AuthenticationProvider authenticationProvider() {
       DaoAuthenticationProvider p = new DaoAuthenticationProvider();
       p.setUserDetailsService(userDetailsService);
       p.setPasswordEncoder(passwordEncoder());
       return p;
   }
}

// Method-level security:
@PreAuthorize("hasRole('ADMIN')")
@DeleteMapping("/{id}")
public void delete(@PathVariable Long id) { }

@PreAuthorize("#userId == authentication.principal.id")   // own data only
@GetMapping("/{userId}/profile")
public Profile get(@PathVariable Long userId) { }
```

Password handling and the mistakes to avoid

```
@Service
class UserService {
   private final PasswordEncoder encoder;    // BCryptPasswordEncoder

   public void register(RegisterRequest req) {
       if (repo.existsByEmail(req.email()))
           throw new DuplicateEmailException();
       User u = new User();
       u.setEmail(req.email());
       u.setPassword(encoder.encode(req.password()));   // HASH, never encrypt
       u.setRole(Role.USER);
       repo.save(u);
   }
}

// BCrypt specifics:
//  - Automatically generates and stores a random salt with the hash
//  - Deliberately slow (a cost factor of 10-12) to defeat brute force
//  - encoder.matches(rawPassword, storedHash) is how you verify
//  - Each hash of the same password differs (different salt) - that's expected

// NEVER:
//   store plain text passwords
//   use MD5 / SHA-1 / SHA-256 alone (too fast -> GPU brute force)
//   implement your own crypto
//   log passwords or put them in a JWT
```

SAY IT LIKE THIS IN THE INTERVIEW

"Spring Security is a filter chain in front of my controllers. For a stateless API I use JWTs: the user logs in, I verify the password with BCrypt, issue a signed token carrying subject, roles and expiry, and a filter validates that token on each request and populates the SecurityContext. I disable CSRF for stateless APIs but configure CORS explicitly, and I use @PreAuthorize for method-level authorization. Passwords are BCrypt-hashed with a per-user salt — never encrypted or stored in plain text."

Flashcards

Q. Authentication vs authorization?
A. Authentication verifies identity (who are you? → 401 if it fails). Authorization verifies permissions (what can you do? → 403 if it fails).

Q. What is a JWT and what's in it?
A. A stateless, signed token: header (algorithm), payload (claims like subject, roles, expiry) and a signature. Base64-encoded — readable by anyone, so never put secrets in it.

Q. Why is JWT called stateless?
A. The server stores no session; all the information is in the token itself, verified cryptographically on each request. That's what makes it scale horizontally.

Q. How do you invalidate a JWT before it expires?
A. You can't easily — that's the main drawback. Use short expiry plus refresh tokens, or maintain a server-side denylist/token version in the DB.

Q. Why BCrypt instead of SHA-256 for passwords?
A. BCrypt is deliberately slow and includes a per-password salt, defeating rainbow tables and GPU brute force. General-purpose hashes are fast, which is exactly wrong for passwords.

Q. CSRF vs CORS?
A. CSRF tricks a browser into sending an authenticated request (mitigated by tokens/same-site cookies). CORS is a browser policy controlling which origins may read responses (configured on the server). They're unrelated problems.

Q. 401 vs 403?
A. 401 Unauthorized = not authenticated (no or bad credentials). 403 Forbidden = authenticated but lacking permission.

Q. What is the Spring Security filter chain?
A. A chain of servlet filters each request passes through before the DispatcherServlet — handling auth, JWT validation, exception translation and authorization.

Quiz

Q1. A user is logged in but tries to access an admin-only page. What status code?

[ ] A. 401 Unauthorized
[x] B. 403 Forbidden
[ ] C. 404 Not Found
[ ] D. 500

> **Why:** They're authenticated but not authorized for that resource → 403. 401 means we don't know who they are.

Q2. Is the JWT payload encrypted?

[ ] A. Yes, fully encrypted
[x] B. No — it's Base64-encoded and readable by anyone; only the signature is cryptographic
[ ] C. Only if you use HS256
[ ] D. It's hashed

> **Why:** JWTs are signed, not encrypted (unless you use JWE). Never put passwords or secrets in the payload.

Q3. How should passwords be stored?

[ ] A. Encrypted with AES so they can be recovered
[x] B. Hashed with BCrypt/Argon2 including a per-user salt
[ ] C. Hashed with MD5
[ ] D. In plain text, but the DB is secure

> **Why:** Passwords are hashed (one-way) with a slow, salted algorithm. Encryption is reversible, which you never want for passwords.

Q4. Which Spring Security setting makes the API stateless?

[ ] A. csrf().disable()
[x] B. SessionCreationPolicy.STATELESS
[ ] C. permitAll()
[ ] D. @PreAuthorize

> **Why:** STATELESS tells Spring Security not to create or use an HTTP session — appropriate when every request carries a JWT. CSRF disable is a related but separate choice.

8. Testing: JUnit 5 & Mockito Core 35 min

Often skipped by freshers, which is exactly why mentioning it stands out. Know the annotations, a basic Mockito test, and `@WebMvcTest` vs `@DataJpaTest` vs `@SpringBootTest`.

Key points

- **JUnit 5** (Jupiter) annotations: `@Test`, `@BeforeEach`, `@AfterEach`, `@BeforeAll`, `@AfterAll`, `@ParameterizedTest`, `@DisplayName`, `@Nested`, `@Disabled`.
- Assertions: `assertEquals`, `assertTrue`, `assertThrows`, `assertAll`, and AssertJ's fluent `assertThat(x).isEqualTo(y)`.
- **Mockito**: `@ExtendWith(MockitoExtension.class)`, `@Mock` for dependencies, `@InjectMocks` for the class under test, `when(...).thenReturn(...)`, `verify(...)`.
- **Test slices**: `@WebMvcTest` (controllers only, with `@MockBean` services), `@DataJpaTest` (repositories with an in-memory DB), `@SpringBootTest` (full context — slowest).
- **Test pyramid**: many unit tests, fewer integration tests, fewest end-to-end tests.
- Mock only what you own — don't mock the class under test, value objects, or (usually) the database in integration tests.

Code

A unit test with JUnit 5 + Mockito

```
@ExtendWith(MockitoExtension.class)      // enables @Mock/@InjectMocks
class OrderServiceTest {

   @Mock OrderRepository repo;            // the dependency is mocked
   @Mock PaymentClient payment;
   @InjectMocks OrderService service;     // real instance, mocks injected

   @Test
   @DisplayName("should place an order and charge the customer")
   void placeOrder_success() {
       // given (arrange)
       Order order = new Order(1L, BigDecimal.TEN);
       when(repo.findById(1L)).thenReturn(Optional.of(order));
       when(payment.charge(any())).thenReturn(true);

       // when (act)
       boolean result = service.placeOrder(1L);

       // then (assert)
       assertThat(result).isTrue();
       verify(payment, times(1)).charge(any());     // interaction check
       verify(repo).save(order);
       verifyNoMoreInteractions(payment);
   }

   @Test
   void placeOrder_missingOrder_throws() {
       when(repo.findById(99L)).thenReturn(Optional.empty());
       assertThrows(ResourceNotFoundException.class, () -> service.placeOrder(99L));
       verify(payment, never()).charge(any());
   }

   @ParameterizedTest
   @ValueSource(ints = {0, -1, Integer.MIN_VALUE})
   void rejectsInvalidQuantities(int qty) {
       assertThatThrownBy(() -> service.addItem(qty))
           .isInstanceOf(IllegalArgumentException.class);
   }
}
```

Testing the web layer and the persistence layer

```
// CONTROLLER slice: fast, no DB, no full context
@WebMvcTest(UserController.class)
class UserControllerTest {
   @Autowired MockMvc mvc;                 // drives HTTP requests in-process
   @MockBean UserService service;          // the service is mocked out

   @Test
   void getUser_returns200() throws Exception {
       when(service.findById(1L))
           .thenReturn(Optional.of(new UserResponse(1L, "Parvez", "p@x.com", 24)));

       mvc.perform(get("/api/users/1"))
          .andExpect(status().isOk())
          .andExpect(jsonPath("$.name").value("Parvez"))
          .andExpect(content().contentType(MediaType.APPLICATION_JSON));
   }

   @Test
   void getMissingUser_returns404() throws Exception {
       when(service.findById(2L)).thenReturn(Optional.empty());
       mvc.perform(get("/api/users/2")).andExpect(status().isNotFound());
   }
}

// REPOSITORY slice: runs against an embedded H2 with the real schema
@DataJpaTest
class UserRepositoryTest {
   @Autowired TestEntityManager em;
   @Autowired UserRepository repo;

   @Test
   void findByEmail_works() {
       em.persistAndFlush(new User("Parvez", "p@x.com"));
       assertThat(repo.findByEmail("p@x.com")).isPresent();
   }
}

// FULL integration test
@SpringBootTest(webEnvironment = WebEnvironment.RANDOM_PORT)
@ActiveProfiles("test")
class OrderApiIntegrationTest {
   @Autowired TestRestTemplate rest;
   @Test void fullFlow() { /* real HTTP against a running server */ }
}
```

AssertJ assertions and the naming convention

```
// Prefer AssertJ - fluent and far more readable than raw JUnit assertions
assertThat(name).isEqualTo("Parvez");
assertThat(list).hasSize(3).contains("a").doesNotContain("z");
assertThat(list).extracting(User::getName).containsExactly("a", "b", "c");
assertThat(map).containsKey("k").containsEntry("k", 1);
assertThatThrownBy(() -> service.run())
       .isInstanceOf(IllegalStateException.class)
       .hasMessageContaining("not initialised");
assertThat(optional).isPresent().get().extracting(User::getAge).isEqualTo(24);

// Naming: methodUnderTest_condition_expectedResult
//   placeOrder_validRequest_returnsCreated
//   findById_unknownId_throwsNotFound
//   withdraw_insufficientFunds_throwsException
// Or BDD style: givenX_whenY_thenZ

// Coverage target: high on business logic (services), don't chase 100% overall -
// testing getters/setters/framework code wastes effort.
```

SAY IT LIKE THIS IN THE INTERVIEW

"I write JUnit 5 tests with Mockito: @Mock the dependencies and @InjectMocks the class under test, then given/when/then with AssertJ's fluent assertions. For Spring layers I use test slices — @WebMvcTest with

MockMvc for controllers and @DataJpaTest for repositories — because they're far faster than loading the full context with @SpringBootTest. The pyramid I aim for is lots of unit tests, some integration tests, and a few end-to-end tests."

Flashcards

Q. @BeforeEach vs @BeforeAll?
A. @BeforeEach runs before every test method; @BeforeAll runs once before all tests in the class (and must be static unless the class is @TestInstance(PER_CLASS)).

Q. @Mock vs @InjectMocks?
A. @Mock creates a mock of a dependency. @InjectMocks creates a real instance of the class under test and injects the mocks into it.

Q. @WebMvcTest vs @DataJpaTest vs @SpringBootTest?
A. @WebMvcTest loads only the web layer (use @MockBean for services). @DataJpaTest loads JPA against an embedded DB. @SpringBootTest loads the whole context — slowest, used for integration tests.

Q. @Mock vs @MockBean?
A. @Mock is a plain Mockito mock (unit tests). @MockBean is Spring Boot's — it adds or replaces a bean in the Spring context (used in slice tests).

Q. What is MockMvc?
A. A Spring test utility that drives HTTP requests against your controllers without starting a real server, letting you assert status codes and JSON responses.

Q. What does verify() do in Mockito?
A. It asserts that an interaction happened — e.g. verify(mock, times(1)).method(args), verify(..., never()), verifyNoMoreInteractions().

Q. What is the test pyramid?
A. Many fast unit tests at the base, fewer integration tests in the middle, and few slow end-to-end/UI tests at the top.

Q. What should you NOT mock?
A. The class under test, simple value objects/DTOs, and (in integration tests) the database — mocking your own DB hides real query bugs.

Quiz

Q1. Which annotation injects a mock into the class you're testing?

[ ] A. @Mock
[x] B. @InjectMocks
[ ] C. @Autowired
[ ] D. @Spy

Why: @InjectMocks creates a real instance and injects the @Mock dependencies into it (via constructor, setter or field).

Q2. Which test slice is fastest for testing a controller in isolation?

[ ] A. @SpringBootTest
[x] B. @WebMvcTest
[ ] C. @DataJpaTest
[ ] D. @ContextConfiguration

Why: @WebMvcTest loads only the web layer and lets you @MockBean the service, so no DB or full context startup.

Q3. What does `verify(mock, times(1)).save(any())` assert?

[ ] A. The method returned once
[x] B. The mock's save method was called exactly once with any argument
[ ] C. The save took one second
[ ] D. The mock was created once

Why: verify checks interactions with the mock — useful when the method under test returns void and there's no return value to assert.

Q4. In JUnit 5, which annotation replaces JUnit 4's @Before?

[ ] A. @BeforeAll
[x] B. @BeforeEach
[ ] C. @Setup
[ ] D. @Initialize

Why: JUnit 5 (Jupiter) renamed it @BeforeEach; @BeforeAll replaces @BeforeClass and must be static by default.

9. Microservices & Architecture Overview Edge 30 min

Freshers aren't expected to run microservices, but they ARE expected to know the vocabulary and the trade-off. Being able to say "monolith first" correctly is a strong signal.

Key points

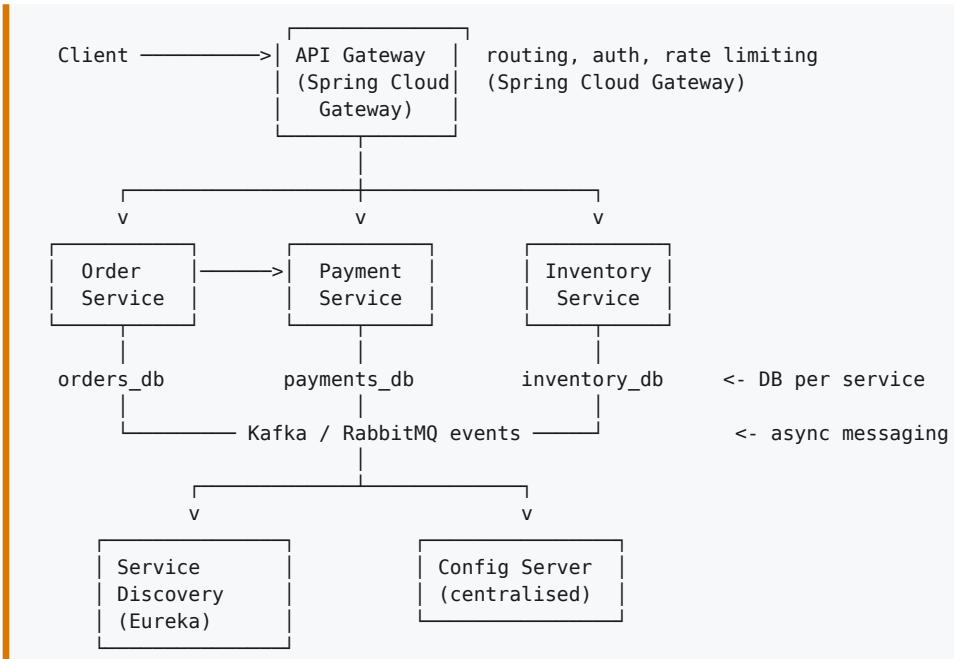
- **Monolith**: one deployable unit, shared DB. Simple to build, test and deploy; hard to scale teams and parts independently.
- **Microservices**: independently deployable services, each owning its data, communicating over HTTP (REST/gRPC) or messaging (Kafka/RabbitMQ).
- **Trade-offs**: microservices buy independent scaling, deployment and tech choice — at the cost of network failures, distributed transactions, debugging complexity and ops overhead. Start with a monolith unless you have a real reason.
- Key patterns: **API Gateway** (single entry point: routing, auth, rate limiting), **Service Discovery** (Eureka/Consul/K8s DNS), **Config Server** (centralised config), **Circuit Breaker** (Resilience4j — fail fast), **Distributed tracing** (Sleuth/Zipkin), **Centralised logging** (ELK).
- **CAP theorem**: in a network partition you must choose Consistency or Availability. (PACELC adds: else you trade latency vs consistency.)
- **12-factor app**: config in the environment, stateless processes, logs as event streams, etc.

Reference

Concern	Monolith	Microservices
Deployment	One unit, all or nothing	Independent per service
Scaling	Scale the whole app	Scale only the hot service
Data	One shared database	Database per service
Transactions	ACID, simple	Sagas / eventual consistency
Complexity	In the code	In the infrastructure
Best for	Small teams, early products	Large systems, many teams, uneven load

Code

What a typical Spring Cloud setup looks like




```
Resilience4j circuit breaker  -> fail fast when a downstream service is down
Spring Cloud Sleuth / Micrometer -> trace IDs across services
Actuator + Prometheus + Grafana  -> metrics and health
```

Service-to-service calls and resilience

```
// Declarative REST client (Spring Cloud OpenFeign)
@FeignClient(name = "payment-service", fallback = PaymentFallback.class)
public interface PaymentClient {
   @PostMapping("/api/payments")
   PaymentResponse charge(@RequestBody PaymentRequest req);
}

// Or the modern HTTP interface (Spring 6)
@HttpExchange("/api/payments")
public interface PaymentApi {
   @PostExchange PaymentResponse charge(@RequestBody PaymentRequest req);
}

// Circuit breaker: stop calling a failing service so it can recover
@CircuitBreaker(name = "paymentService", fallbackMethod = "chargeFallback")
@Retry(name = "paymentService")
@TimeLimiter(name = "paymentService")
public CompletableFuture<PaymentResponse> charge(PaymentRequest req) {
   return CompletableFuture.supplyAsync(() -> client.charge(req));
}
public CompletableFuture<PaymentResponse> chargeFallback(PaymentRequest req, Throwable t) {
   return CompletableFuture.completedFuture(PaymentResponse.pending());
}

// Saga pattern: instead of a distributed transaction, each service commits
// locally and publishes an event; compensating actions undo earlier steps
// if a later step fails (e.g. cancel order -> refund payment -> restock).
```

How to answer 'monolith or microservices?'

```
A strong answer:
"It depends on the problem and the team. I'd start with a well-structured
monolith — modular packages, clear boundaries, no shared mutable state —
because it's far cheaper to build, test, debug and deploy. I'd move to
microservices when there's a specific pressure: one part needs to scale
independently, teams are stepping on each other's deploys, or different
parts need different technology or availability guarantees.

If we did go distributed, each service would own its database, communicate
over REST for queries and events for state changes, sit behind an API
gateway, and use circuit breakers plus centralised logging and tracing —
because the hard part isn't splitting the code, it's handling partial
failure, eventual consistency and observability."

Red flags to avoid:
 "Microservices are always better"
 "Let's split it into 20 services on day one"
 Sharing one database across services (a distributed monolith)
```

SAY IT LIKE THIS IN THE INTERVIEW

"I'd start with a modular monolith and only split into microservices for a concrete reason — independent scaling, team autonomy or different availability needs. If we go distributed, each service owns its data, they communicate via REST or events, sit behind an API gateway, and need circuit breakers, centralised logging and distributed tracing, because partial failure and eventual consistency are the real costs."

Flashcards

Q. Monolith vs microservices — the core trade-off?
A. Monoliths are simpler to build, test, debug and deploy. Microservices give independent scaling, deployment and team autonomy, but add network failures, distributed data problems and operational complexity.

Q. What is an API Gateway?
A. A single entry point for clients that handles routing, authentication, rate limiting, CORS and request aggregation — so those concerns aren't duplicated in every service.

Q. What is a circuit breaker?
A. A resilience pattern (Resilience4j) that stops calling a failing downstream service and returns a fallback, giving it time to recover and preventing cascading failures.

Q. What is service discovery?
A. A registry (Eureka, Consul, Kubernetes DNS) where services register themselves and look each other up by name instead of hard-coded host:port.

Q. What is the Saga pattern?
A. A way to manage data consistency across services without distributed transactions: each service commits locally and emits an event, with compensating transactions to roll back if a later step fails.

Q. What is eventual consistency?
A. The system may be briefly inconsistent while updates propagate, but converges to a consistent state — the normal trade-off when each service owns its own database.

Q. State the CAP theorem.
A. During a network partition, a distributed system must choose between Consistency (all nodes see the same data) and Availability (every request gets a response) — it can't guarantee both.

Q. What is distributed tracing?
A. Propagating a trace ID across service calls so you can follow one request end to end (Sleuth/Micrometer + Zipkin or Jaeger).

Q. What's the database-per-service rule?
A. Each microservice owns its data and no other service reads it directly — sharing a database creates a distributed monolith and couples deployments.

Quiz

Q1. What is the biggest hidden cost of microservices?

[ ] A. They're harder to code

[x] B. Operational complexity: network failures, distributed data, debugging and observability

[ ] C. Java doesn't support them

[ ] D. They cost more in licences

> **Why:** The code is easy to split; the difficulty is partial failure, eventual consistency, tracing a request across services, and deploying/monitoring many moving parts.

Q2. What does a circuit breaker do when a downstream service is down?

[ ] A. Retries forever

[x] B. Opens and returns a fallback immediately, giving the service time to recover

[ ] C. Deletes the service

[ ] D. Switches to a backup database

> **Why:** After failures exceed a threshold the circuit opens; calls short-circuit to the fallback instead of piling on load, then it half-opens to test recovery.

Q3. Two services must both succeed or neither should. In microservices you'd typically use:

[ ] A. A distributed transaction (2PC)

[x] B. The Saga pattern with compensating actions

[ ] C. A shared database

[ ] D. synchronized blocks

> **Why:** 2PC doesn't scale and couples availability. Sagas commit locally per service and publish events, with compensating transactions to undo on failure.

Q4. According to CAP, during a network partition you must choose:

[x] A. Consistency or Availability

[ ] B. Speed or Security

[ ] C. SQL or NoSQL

[ ] D. REST or gRPC

> **Why:** Partition tolerance isn't optional in a distributed system, so the real choice is C or A during a partition.

10. Explaining Your Project (the round that decides it) Must-know 40 min

For a fresher, "tell me about your project" is the highest-leverage question in the whole interview. Most candidates ramble. You'll use a structure — and rehearse it out loud today, not tomorrow morning.

Key points

- Use **30-60 seconds** for the overview, then let questions drive the detail. Rambling for five minutes loses the room.
- Structure: **What** it does → **Who** uses it → **Tech stack** → **Your specific contribution** → **A challenge you solved** → **What you'd improve**.
- Name the *problem* before the features: "The manual process took 2 hours a day, so we automated it."
- Be ready for: "Why this technology?", "What was the hardest bug?", "How did you test it?", "What would you do differently?"
- Prepare numbers: users, records, response time, test coverage, % improvement. Specifics beat adjectives.
- **Never claim something you can't discuss for two minutes.** Every bullet on your resume is a question waiting to be asked.

Code

A 45-second project pitch you can adapt

```
"I built an Order Management System — a REST API that handles the full
order lifecycle: customers place orders, the service validates stock,
processes payment and sends a confirmation email.

The stack is Java 17 with Spring Boot 3, Spring Data JPA on PostgreSQL,
with Spring Security using JWT for authentication. Maven for the build,
JUnit and Mockito for tests, and it's Dockerised.

I owned the order and payment modules end to end. Specifically I designed
the schema, implemented the REST endpoints with DTO validation, and set up
the JPA entity relationships.

The hardest problem was performance: the order history endpoint took 3
seconds because of an N+1 query — one query for the orders and one for
each order's items. I found it by enabling Hibernate SQL logging, fixed it
with a JOIN FETCH and added pagination, which brought it down to under
200ms.

If I were rebuilding it, I'd add integration tests with Testcontainers and
move the email sending to an async queue so the API doesn't wait on SMTP."

// Why this works:
//   1. What it does (2 sentences, no fluff)
//   2. Concrete stack (shows breadth)
//   3. YOUR contribution (ownership, not "we did")
//   4. A real problem + how you diagnosed it + the measurable result
//   5. Self-awareness about improvements (maturity signal)
```

The five follow-up questions and how to handle each

```
Q: "Why did you choose Spring Boot?"
A: "It removes the boilerplate - auto-configuration gave me an embedded
  server, Jackson and a DataSource without manual config, and the starter
  POMs keep dependency versions compatible. It let me focus on features."

Q: "What was the hardest bug you faced?"
A: Use STAR: Situation, Task, Action, Result.
  "The order history page was slow. I enabled SQL logging and saw 200
  queries instead of 1 - a classic N+1 from lazy loading. I fixed it with
  JOIN FETCH plus pagination. Response time went from 3s to 180ms."

Q: "How did you handle security?"
A: "Spring Security with JWT. Passwords are BCrypt-hashed, stateless
  sessions, roles enforced with @PreAuthorize at the method level, and
  CORS configured explicitly. Validation on every request body."

Q: "How did you test it?"
A: "JUnit 5 and Mockito for unit tests on the service layer, @WebMvcTest
  with MockMvc for controllers, @DataJpaTest for repositories. Around
  70% coverage on the business logic."

Q: "What would you do differently?"
A: "Add integration tests with Testcontainers, extract the email sending
  into an async queue, and add Flyway for schema migrations instead of
  ddl-auto=update."
```

Questions YOU should ask them (prepare 3)

```
Good questions signal real interest:
 1. "What does the team's tech stack look like day to day, and which Java
     version are you on?"
 2. "How do you handle code reviews, testing and deployments here?"
 3. "What does success look like for someone in this role in the first
     six months?"
 4. "What's the biggest technical challenge the team is facing right now?"
 5. "Is there anything from my background you'd like me to clarify?"

Avoid in the first round:
 - Salary, leave policy, work-from-home (save these for HR)
 - Anything you could have found on their website

Closing the interview:
 "Thank you — I enjoyed discussing the role. I'm genuinely interested in
  the backend work you described, especially [specific thing]. What are
  the next steps?"
```

SAY IT LIKE THIS IN THE INTERVIEW

Structure beats talent here. Rehearse your 45-second pitch out loud today — problem, stack, your contribution, one hard problem with a measurable result, and one improvement. Then every bullet on your resume should survive two minutes of follow-up questions.

Flashcards

Q. Give the 6-part structure for explaining a project.
A. What it does → Who uses it → Tech stack → Your specific contribution → A challenge and how you solved it → What you'd improve.

Q. What is the STAR method?
A. Situation, Task, Action, Result — a structure for answering behavioural questions with a concrete story and a measurable outcome.

Q. How long should your initial project explanation be?
A. 30-60 seconds. Then let their questions drive the depth — rambling for five minutes loses the room.

Q. What's the strongest thing you can include in a project answer?
A. A specific problem you diagnosed, the technique you used, and a measurable result ("N+1 query, fixed with JOIN FETCH, 3s → 180ms").

Q. What should you never put on your resume?
A. Anything you can't discuss for two minutes. Every bullet is an invitation to ask about it.

Q. What questions should you ask the interviewer?
A. About the stack and Java version, engineering practices (reviews, testing, deploys), what success looks like in six months, and current technical challenges.

Q. How do you answer 'what's your weakness?'
A. Name a real, non-fatal weakness and show what you're doing about it — e.g. "I used to skip writing tests; now I write them alongside the service layer."

Q. What should you do if you don't know an answer?
A. Say so honestly, then reason aloud: "I haven't used that directly, but here's how I'd approach it..." Interviewers rate the reasoning, not the guess.

Quiz

Q1. What is the ideal length for your opening project explanation?

[x] A. 30-60 seconds
[ ] B. 5 minutes
[ ] C. 10 seconds
[ ] D. As long as it takes

Why: Short enough to hold attention, detailed enough to earn follow-ups. Then let their questions drive the depth.

Q2. Which element most strengthens a project answer?

[ ] A. Listing every technology you touched
[x] B. A specific problem you diagnosed, the fix, and a measurable result
[ ] C. Saying the team was great
[ ] D. Mentioning the project was complex

Why: Concrete, measurable problem-solving is what separates a real contributor from someone who was in the room.

Q3. You're asked something you genuinely don't know. Best response?

[ ] A. Guess confidently
[x] B. Say you don't know, then reason through how you'd approach it
[ ] C. Change the subject
[ ] D. Say 'we didn't cover that in training'

Why: Honesty plus visible reasoning scores far better than a bluff. Interviewers are testing how you think under uncertainty.

Q4. What does STAR stand for?

[ ] A. Stack, Tech, Architecture, Result
[x] B. Situation, Task, Action, Result
[ ] C. Start, Test, Analyse, Refactor
[ ] D. Scope, Time, Assignments, Review

Why: STAR is the standard structure for behavioural answers — set the scene, your task, what you did, and the outcome (with numbers if possible).